Create Demo Data

AI Engineer | Machine Learning Engineer | Generative AI

PROFESSIONAL SUMMARY
AI Engineer specializing in designing and deploying end-to-end AI systems across Machine Learning, Computer Vision, and Generative AI. Experienced in building scalable ML and AI pipelines using Python, PyTorch, TensorFlow, FastAPI, Docker, and AWS. Hands-on experience developing medical AI applications, predictive models, RAG architectures, and multi-agent systems, with knowledge of MLOps practices including DVC, MLflow, CI/CD, and cloud deployment.

Technical Projects & Experience

- Machine Learning & Deep Learning: Supervised Learning, Unsupervised Learning, PyTorch, TensorFlow, Transfer Learning, YOLO, OpenCV, Scikit-learn, XGBoost, NumPy, Pandas, Seaborn
- Generative AI & Agentic Frameworks: LangChain, LangGraph, OpenAI API, Gemini, CrewAI, Retrieval-Augmented Generation (RAG), Multi-Agent Systems
- MLOps & Cloud Deployment: MLflow, Dagshub, DVC, Docker, FastAPI, GitHub Actions, CI/CD, AWS (EC2, S3), Google Cloud Platform (GCP), Git, Linux, Bash, n8n
- Databases & Vector Search: SQL, MongoDB, Pinecone, ChromaDB

TECHNICAL PROJECTS & EXPERIENCE

Self-Corrective RAG System with LangGraph

Self-Corrective RAG System with LangGraph - 🔗 GitHub
Python | LangGraph | LangChain | OpenAI API | FAISS | Streamlit | Docker | AWS



- Built a **self-corrective RAG system** with LangGraph implementing query routing, adaptive retrieval, relevance evaluation, answer generation, hallucination/groundedness checking, and iterative query rewriting.
- Developed a **state-driven agentic workflow** with conditional routing and retry loops to improve retrieval quality and reduce unsupported answers.
- Built a Streamlit application with **persistent chat/project memory, document ingestion, FAISS vector search, OpenAI embeddings, and streaming graph execution**.
- Containerized and deployed the application using **Docker, AWS ECR/EC2, and GitHub Actions CI/CD**.

Multi-Modal Retrieval-Augmented Generation (RAG) System

Multi-Modal Retrieval-Augmented Generation (RAG) System - 🔗 GitHub
Role: AI Engineer | Tech: OpenAI Vision, Pinecone, Python, LangChain, AWS

- Architected a Multimodal RAG system capable of extracting and reasoning over text, complex tables, and embedded images inside unstructured PDF documents.
- Built a vision-aware semantic search pipeline using Pinecone and vision-LLMs to preserve document layout context.
- Reduced retrieval hallucinations by 25% using a hybrid retrieval strategy combining vector embeddings with cross-modal image-to-text descriptions.

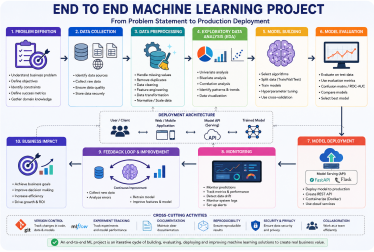
End to End Deep learning PneumoScan-AI

PneumoScan-AI Automated Chest X-ray Segmentation System - 🔗 GitHub
Role: AI Developer | Tech: TensorFlow, U-Net, ResNet50, Computer Vision, OpenCV

- Developed an automated lung segmentation and abnormality detection model using U-Net with a ResNet50 backbone for chest X-ray analysis.
- Applied transfer learning and extensive data augmentation, achieving a Dice similarity coefficient above 0.80 across varying image conditions.
- Engineered automated highlighting features to identify potential abnormal lung regions for medical image analysis.

End to End Machine learing

Role: ML Engineer | Tech: Python, Numpy, Pandas, Seaborn, Scikit-learn, FastAPI, Docker, AWS



- A machine learning model to predict shipment costs based on factors such as distance, weight, delivery type, shipping method, and other logistics features. Performed data preprocessing, feature engineering, model training, and hyperparameter tuning to build an accurate price prediction system.
- Developed an end-to-end machine learning system for shipment price prediction with real-time inference capabilities.
- Containerized the application using Docker and deployed the API on AWS infrastructure.

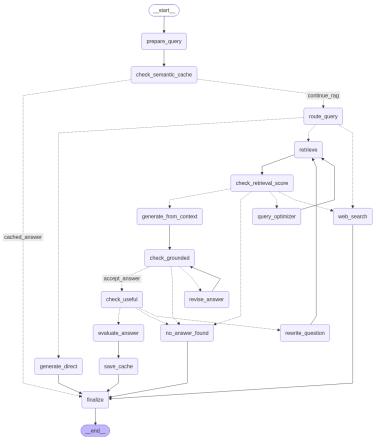
EDUCATION

Bachelor of Science (BSc)
Narsingdi Government College, Bangladesh

LangGraph-DocuMind-AWS-Deployment

LangGraph DocuMind is an Advanced Self-Corrective Multimodal Retrieval-Augmented Generation (RAG) system that can answers questions over your own documents (text, tables and images from PDFs) and falls back to live web search when the question needs fresh information. It intelligently validates and refines doc-grounded answers through real-time state tracking, persistent chat history, and automated source citations.

Architecture



Nodes

Node	What it does
`prepare_query`	Resets all per-turn state, turns an uploaded image into a text description, and rewrites follow-up questions into standalone questions using chat history.
`check_semantic_cache`	Looks for a similar, fresh, same-project answer. Skipped for image queries and when `skip_cache` is set.
`route_query`	Classifies the question as `rag`, `tool` (web) or `direct`. Defaults to `rag` if the classifier fails.
`retrieve`	Vector search (`similarity_search_with_score`) followed by hybrid reranking (70% vector score, 30% keyword coverage).
`check_retrieval_score`	Keeps chunks that pass the vector threshold; rescues the best keyword match if none do. Handles both distance (FAISS/ Chroma) and similarity (Pinecone) scores.
`query_optimizer`	Rewrites the query for retrieval (once) and loops back to `retrieve`.
`generate_from_context`	Answers from the retrieved chunks and attaches sources, citations, and images/tables found in those chunks.

Node	What it does
`check_grounded`	Strict verifier: is every claim supported by the context? (`fully / partially / no support`)
`revise_answer`	Rewrites the answer as direct quotes from the context, then re-checks groundedness.
`check_useful`	Does the answer actually address the question?
`rewrite_question`	Rewrites the retrieval query and restarts retrieval with a fresh retry budget.
`evaluate_answer`	Computes hallucination, groundedness and overall quality metrics.
`save_cache`	Stores the answer (with sources, citations and media) in the semantic cache, only if marked reliable.
`web_search`	Tavily search with images, then an answer written strictly from the results.
`generate_direct`	Plain LLM answer for general knowledge questions.
`no_answer_found`	Safe fallback when nothing relevant or supported is found.
`finalize`	Appends the turn to chat history; every path ends here.

Tech stack

Area	Technology
Orchestration	LangGraph, LangChain
LLM / embeddings	OpenAI (`text-embedding-3-small`, 1536 dimensions; chat model configurable)
Vector database	Pinecone (serverless) or FAISS (local)
Web search	Tavily through an MCP client
PDF extraction	pdfplumber (text, tables), PyMuPDF (images)
Conversation memory	LangGraph checkpointer with SQLite (`AsyncSqliteSaver`)
Observability	LangSmith tracing, structured JSON query logs

Area	Technology
API (planned)	FastAPI with Server-Sent Events

Response format

After a turn, the final state contains:

Field	Description
`answer`	Final answer text
`sources`	Document names or web URLs used
`citations`	`[{source, page, snippet}]`
`media`	Images, charts and tables to display (see below)
`confidence_score`	Retrieval confidence adjusted by the groundedness verdict
`evaluation`	Hallucination, groundedness and quality metrics
`route`	`rag`, `tool` or `direct`
`cache_hit`	Whether the answer came from the semantic cache

`media` items:

Self Employed Company

Employee ID	Name	Department	Role	Age	Salary	Experience_Years	Performance
EMP001	Kaif khan	Engineering	Senior Backend Developer	34	118000	9	Exceeds Expectations
EMP002	Yasin Ahmed	Data & Analytics	Data Scientist	29	95000	5	Exceeds Expectations
EMP003	Maya Khan	Marketing	Content Strategist	27	68000	3	Meets Expectations
EMP004	James Wilson	Sales	Account Executive	41	105000	14	Exceptional
EMP005				31	72000	6	

Employee ID	Name	Department	Role	Age	Salary	Experience_Years	Performance
	Priya Patel	Human Resources	HR Operations Specialist				Meets Expectations
EMP006	David Kim	Product	Principal Product Manager	45	145000	18	Exceptional
EMP007	Jasica Taylor	Finance	Senior Financial Analyst	38	110000	12	Exceeds Expectations
EMP008	Alex RR	Customer Support	Technical Support Lead	26	58000	4	Meets Expectations
EMP009	Oliver Smith	Engineering	Frontend Engineer	25	82000	2	Needs Improvement
EMP010	kavin ss	Design	Lead UX/ UI Designer	36	125000	11	Exceeds Expectations